MODULE 3

COMBINING CLASSIFIERS – VOTING

Wine Quality Classification Using Machine Learning

Submitted by: Mamatha M
USN: 4GW23CI025
Class/Semester: 7th sem
Subject: Machine learning - 2
Department: CSE(AI&ML)

1. OBJECTIVE

The main objective of this assignment is to study and implement ensemble learning using voting classifiers.

The objectives are:

- To use an open wine quality dataset for classification.
- To train three different machine learning classifiers.
- To implement hard voting and soft voting using scikit-learn.
- To compare individual classifiers with ensemble classifiers.
- To evaluate model performance using accuracy, precision, recall, F1-score, and confusion matrices.
- To identify the advantages and limitations of voting classifiers.

2. DATASET DESCRIPTION

I. Dataset source

The White Wine Quality dataset is obtained from the UCI Machine Learning Repository. It contains physicochemical measurements of white wine samples and their corresponding quality scores.

Dataset URL: https://archive.ics.uci.edu/dataset/186/wine+quality

II. Dataset details

- Total number of samples: 4,898.
- Number of input features: 11.
- Number of original columns: 12, including the quality score.
- Data type: Numerical.
- Problem type: Binary classification after target conversion.

The dataset contains the following 11 input features:

- **Fixed acidity:** Measures the non-volatile acids present in wine.
- **Volatile acidity:** Measures volatile acids that can affect the taste of wine.
- **Citric acid:** Indicates the amount of citric acid present in wine.
- **Residual sugar:** Measures the sugar remaining after fermentation.
- **Chlorides:** Indicates the amount of salt present in wine.
- **Free sulfur dioxide:** Measures the free sulfur dioxide available to help protect wine from oxidation and microbial growth.
- **Total sulfur dioxide:** Measures the total amount of free and bound sulfur dioxide.
- **Density:** Measures the density of the wine.
- **pH:** Indicates the acidity or alkalinity of the wine.
- **Sulphates:** Measures sulphate content, which can influence wine properties.
- **Alcohol:** Indicates the alcohol content of the wine.

III. Target definition

The original dataset contains wine quality scores. In this assignment, the original quality score is converted into a binary target.

- Quality score below 7: Not Good (class 0).
- Quality score of 7 or above: Good (class 1).

The model predicts the target class using the 11 physicochemical features.
The original quality column is removed from the input features to avoid target leakage.

IV. Dataset loading

The dataset is loaded using the Pandas library. The data is separated into input features and the target variable. The dataset is divided into training and testing sets using an 80:20 ratio.

3. METHODOLOGY

The following steps are used to implement the wine quality classification system.

Step 1: Import libraries

Pandas and NumPy are used for data handling. Matplotlib and Seaborn are used for visualization. Scikit-learn provides the machine learning models, preprocessing methods, and evaluation metrics.

Step 2: Load and inspect the dataset

The dataset is downloaded from the UCI repository. The first few rows, column names, missing values, and quality distribution are examined.

Step 3: Define the target variable
The original quality score is converted into two classes: Not Good and Good. This transforms the problem into binary classification.

Step 4: Split the dataset
The dataset is divided into 80% training data and 20% testing data. The random state is set to 42 for reproducibility, and stratified splitting is used to preserve the class proportions.

Step 5: Preprocess the data
StandardScaler is applied to the input features used by Logistic Regression. A pipeline ensures that scaling is learned from the training data only. Decision Tree and Random Forest use the original numerical features.

Step 6: Train individual classifiers
Three classifiers are trained:

- Logistic Regression
- Decision Tree Classifier
- Random Forest Classifier

Their predictions are evaluated separately.

Step 7: Apply hard voting
Hard voting combines the predicted class labels from the three classifiers. The class receiving the majority of votes is selected as the final prediction.

Step 8: Apply soft voting
Soft voting combines the predicted probabilities from the three classifiers. The probabilities for each class are averaged, and the class with the highest average probability is selected.

Step 9: Evaluate and compare
All five models are evaluated on the same test dataset. Accuracy, precision, recall, F1-score, and confusion matrices are used to compare their performance.

4. IMPLEMENTATION
The assignment is implemented in Python using Google Colab and the scikit-learn library.

4.1 Importing the libraries
The required libraries are imported to perform data analysis, model training, ensemble learning, and evaluation.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, VotingClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
```

4.2 Loading the dataset and defining the target

```
url = (
    "https://archive.ics.uci.edu/ml/"
    "machine-learning-databases/wine-quality/"
    "winequality-white.csv"
)

df = pd.read_csv(url, sep=";")

df["target"] = (df["quality"] >= 7).astype(int)

X = df.drop(columns=["quality", "target"])
y = df["target"]
```

The dataset is loaded into a Pandas DataFrame. The target variable is created using the original quality score. The original quality column is removed from the input features.

4.3 Splitting the dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

The training data is used to fit the classifiers, while the testing data is used to evaluate their predictions.

4.4 Creating the individual classifiers

```
logistic = make_pipeline(
    StandardScaler(),
```

```
    LogisticRegression(max_iter=2000, random_state=42)
)

decision_tree = DecisionTreeClassifier(
    max_depth=8,
    random_state=42
)

random_forest = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    class_weight="balanced"
)
```

Logistic Regression is a linear classification model. Decision Tree learns decision rules, while Random Forest combines multiple decision trees to make predictions.

4.5 Training and evaluating the individual classifiers

```
models = {
    "Logistic Regression": logistic,
    "Decision Tree": decision_tree,
    "Random Forest": random_forest
}

results = []
predictions = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    predictions[name] = y_pred

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(
            y_test, y_pred, zero_division=0
        ),
        "Recall": recall_score(
            y_test, y_pred, zero_division=0
        ),
        "F1-score": f1_score(
            y_test, y_pred, zero_division=0
        )
    })
```

Each classifier is trained on the training data. Its predictions are compared with the actual test labels, and the evaluation metrics are calculated.

4.6 Implementing hard voting and soft voting

```
estimators = [
```

```
    ("lr", logistic),
    ("dt", decision_tree),
    ("rf", random_forest)
]

hard_voting = VotingClassifier(
    estimators=estimators,
    voting="hard"
)

soft_voting = VotingClassifier(
    estimators=estimators,
    voting="soft"
)

for name, model in [
    ("Hard Voting", hard_voting),
    ("Soft Voting", soft_voting)
]:
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    predictions[name] = y_pred

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(
            y_test, y_pred, zero_division=0
        ),
        "Recall": recall_score(
            y_test, y_pred, zero_division=0
        ),
        "F1-score": f1_score(
            y_test, y_pred, zero_division=0
        )
    })

results_df = pd.DataFrame(results)
print(results_df.round(4))
```

The VotingClassifier combines the three individual classifiers. Hard voting uses class labels, while soft voting uses predicted class probabilities. The results are stored in a table for comparison.

5. Results and Performance Evaluation

I. Performance Comparison of Individual and Ensemble Models

The five models were evaluated using four performance metrics: Accuracy, Precision, Recall, and F1-score. The results obtained from the Google Colab notebook are shown below.

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.8010	0.5842	0.2783	0.3770
Decision Tree	0.8286	0.6447	0.4623	0.5385
Random Forest	0.8867	0.8633	0.5660	0.6838
Hard Voting	0.8469	0.7672	0.4198	0.5427
Soft Voting	0.8582	0.7744	0.4858	0.5971

II. Observations:

- Random Forest achieved the highest accuracy of **88.67%** among all five models.
- Logistic Regression achieved an accuracy of 80.10%, which was the lowest among the tested models.
- Decision Tree performed better than Logistic Regression in all four reported metrics.
- Hard Voting achieved an accuracy of 84.69%, while Soft Voting achieved 85.82%.
- Soft Voting performed better than Hard Voting in accuracy, precision, recall, and F1-score.
- Random Forest achieved the highest precision and F1-score, indicating strong overall performance in identifying good-quality wine.

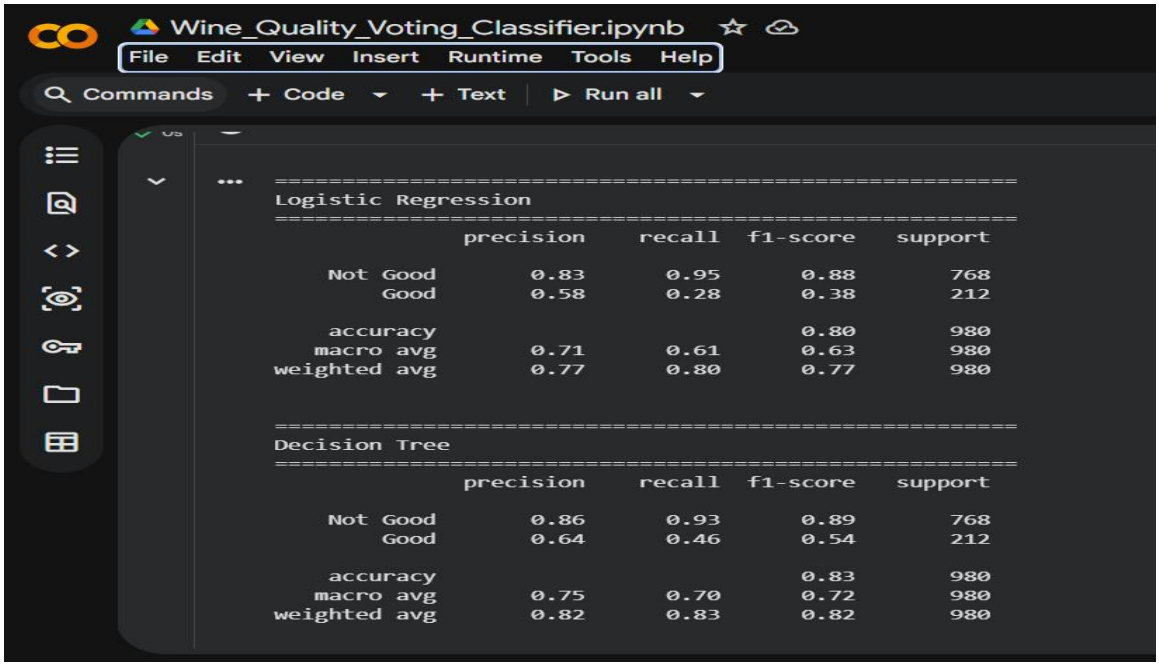
Confusion Matrix Analysis
A confusion matrix was generated for each model to compare its predicted classifications with the actual classifications.

The confusion matrices contain four values:

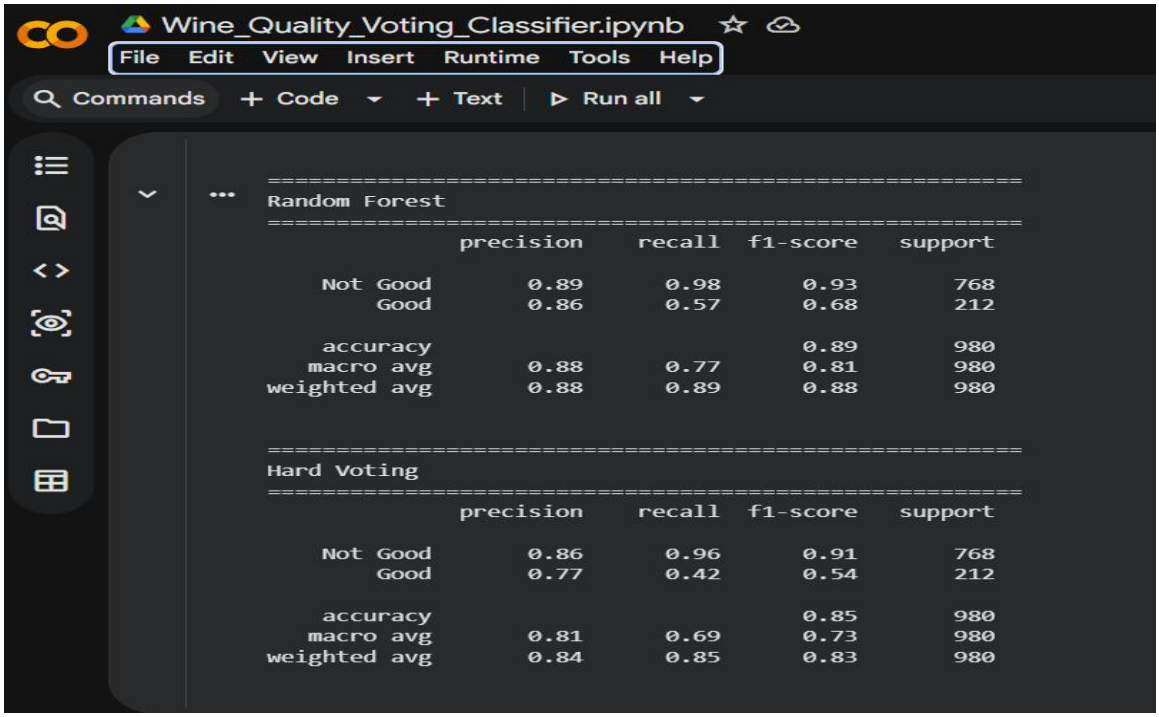
- **True Positive (TP):** Good-quality wine correctly predicted as Good.
- **True Negative (TN):** Not Good wine correctly predicted as Not Good.
- **False Positive (FP):** Not Good wine incorrectly predicted as Good.
- **False Negative (FN):** Good wine incorrectly predicted as Not Good.

The test dataset contained 980 samples, including 768 Not Good samples and 212 Good samples.

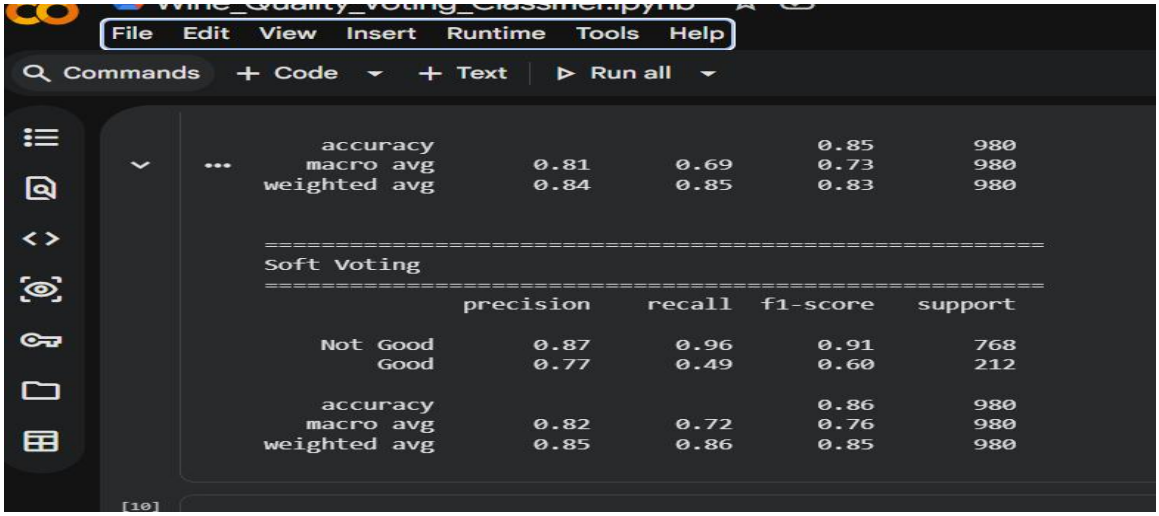
The confusion matrices help identify which types of classification errors each model makes. In this experiment, the models generally performed better at identifying Not Good wine than Good wine.



Confusion matrices for Logistic Regression, Decision Tree



Confusion matrices for Random Forest, Hard Voting



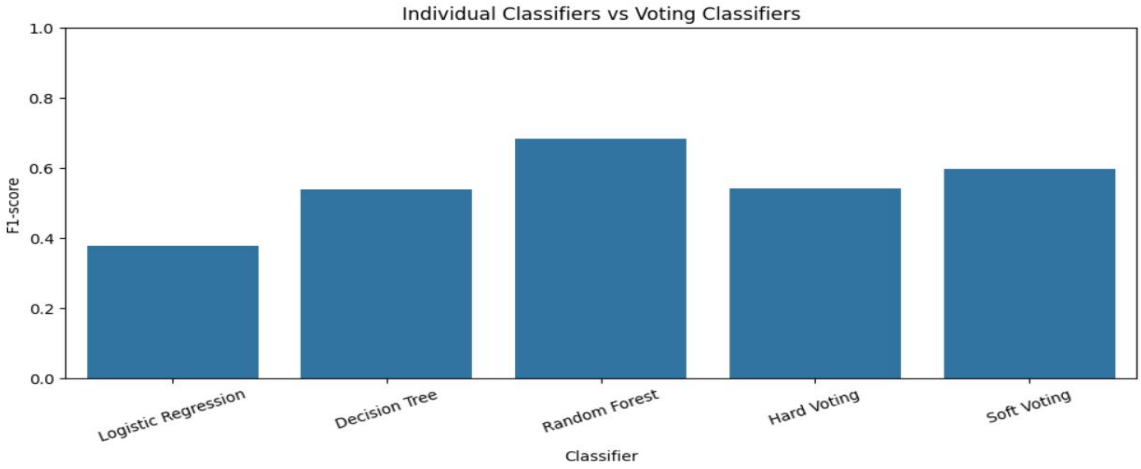
Confusion matrices for Soft Voting.

III. F1-score Comparison

A bar chart was generated to compare the F1-scores of the five models.
The F1-scores were:

- Logistic Regression: 0.3770
- Decision Tree: 0.5385
- Random Forest: 0.6838
- Hard Voting: 0.5427
- Soft Voting: 0.5971

Random Forest achieved the highest F1-score of 0.6838. Soft Voting achieved a higher F1-score than Hard Voting, showing that combining predicted class probabilities performed better than majority voting in this experiment.



F1-score comparison of individual classifiers and voting classifiers.

6. Discussion

The experimental results show that different classification algorithms perform differently on the wine-quality dataset.

Logistic Regression achieved an accuracy of 80.10%, but its recall for the Good class was relatively low. The Decision Tree improved the accuracy to 82.86% and the F1-score to 0.5385.

Random Forest produced the best overall results, with 88.67% accuracy and an F1-score of 0.6838. It was more effective than the other tested models at identifying the Good class.

The ensemble methods also produced useful results. Hard Voting achieved 84.69% accuracy, whereas Soft Voting achieved 85.82%. Soft Voting performed better because it considers the class probabilities predicted by the individual classifiers before selecting the final class.

However, neither voting method outperformed Random Forest in this experiment. This shows that combining classifiers does not always improve performance. The final result depends on the individual models, their predictions, and the dataset.

The recall for the Good class was lower than the recall for the Not Good class across all five models. This indicates that some Good-quality wine samples were incorrectly classified as Not Good.

7. Conclusion

This experiment demonstrated the use of individual classifiers and ensemble learning techniques for wine-quality classification.

Three individual classifiers—Logistic Regression, Decision Tree, and Random Forest—were trained and evaluated. Hard Voting and Soft Voting were then applied to combine their predictions.

The results showed that Random Forest achieved the highest accuracy of 88.67% and the highest F1-score of 0.6838. Soft Voting performed better than Hard Voting, achieving 85.82% accuracy compared with 84.69%.

Although the voting classifiers improved on some individual models, they did not outperform Random Forest. Therefore, the best-performing model in this experiment was Random Forest.

The experiment helped demonstrate how individual classifiers can be compared with ensemble methods using accuracy, precision, recall, F1-score, and confusion matrices.

8. Dataset Attribution and License

The dataset used in this experiment is the **Wine Quality dataset**, obtained from the UCI Machine Learning Repository.

Dataset citation:

Cortez, P., Cerdeira, A., Almeida, F., Matos, T., & Reis, J. (2009). *Wine Quality* [Dataset]. UCI Machine Learning Repository.

Dataset DOI: https://doi.org/10.24432/C56S3T

License: Creative Commons Attribution 4.0 International (CC BY 4.0).

The dataset contains physicochemical measurements of wine and its quality rating. For this experiment, the original quality rating was converted into a binary target: quality below 7 was labelled Not Good (0), and quality of 7 or above was labelled Good (1).